

Part 1: Fundamentals and Analysis

1. What is DSA?

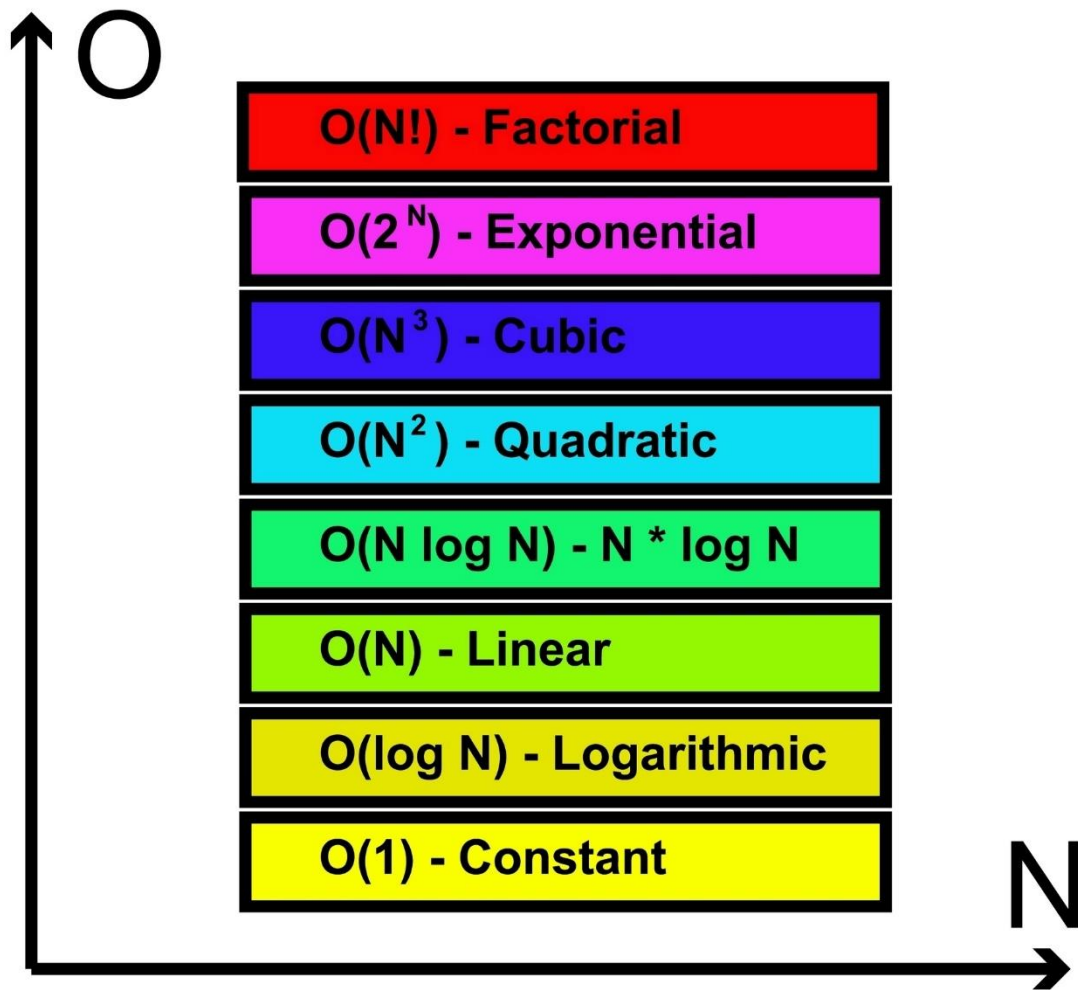
- **Data Structure (DS):** A specific way of organizing and storing data in a computer to be used efficiently.
- **Algorithm (A):** A step-by-step procedure or formula for solving a problem, especially one involving a repetitive operation.

2. Analysis of Algorithms (Complexity)

This measures how the performance of an algorithm changes as the input size (n) grows.

Big O Notation (O):

Describes the **Worst-Case** running time/space complexity. It represents the upper bound of the growth rate.



Big O Notation

Shutterstock

Notation	Name	Description	Example
$O(1)$	Constant	Time/Space is the same regardless of input size.	Accessing an Array element by index.
$O(\log n)$	Logarithmic	Time/Space grows very slowly (input size is cut in half).	Binary Search.
$O(n)$	Linear	Time/Space is directly proportional to the input size.	Linear Search, Traversing a Linked List.

Notation	Name	Description	Example
$O(n \log n)$	Log-Linear	Optimal time complexity for sorting.	Merge Sort, Quick Sort (Average).
$O(n^2)$	Quadratic	Time/Space is proportional to the square of the input size.	Nested loops (Bubble Sort, Selection Sort).
$O(2^n)$	Exponential	Time/Space doubles with every addition to the input set.	Solving the Traveling Salesperson problem (Brute force).

Part 2: Core Data Structures

Data structures are typically categorized as **Linear** (elements arranged sequentially) or **Non-Linear** (elements arranged hierarchically or in a network).

A. Linear Data Structures

Structure	Description	Key Operations	Time Complexity (Search, Insertion, Deletion)	Use Cases
Array	Fixed-size collection of same-type elements stored in contiguous memory locations, accessed by index.	Access, Search, Insert, Delete.	$O(1)$, $O(n)$, $O(n)$, $O(n)$	Fixed-size data, lookups (e.g., Image processing).
Linked List	Collection of nodes where each node contains data and a pointer to the next node.	Traversal, Insert (Head), Delete (Head).	$O(n)$, $O(1)$, $O(1)$	Dynamic data size, implementing Stacks/Queues.
Stack	LIFO (Last In, First Out). Operations are only at the Top .	push() (insert), pop() (delete), peek() (top element).	$O(1)$ for all	Function call stack, Undo/Redo mechanisms, Expression Evaluation.

Structure	Description	Key Operations	Time Complexity (Search, Insertion, Deletion)	Use Cases
Queue	FIFO (First In, First Out) . Operations at the Front (dequeue) and Rear (enqueue).	enqueue() (insert), dequeue() (delete), front() (first element).	$O(1)$ for all	Task scheduling, BFS algorithm, handling requests.

B. Non-Linear Data Structures

Structure	Description	Key Operations	Time Complexity (Avg.)	Use Cases
Hash Table	Uses a Hash Function to map keys to indices in an array. Aims for $O(1)$ access.	Insertion, Search, Deletion.	$O(1)$ (Average)	Dictionaries/Maps, Database indexing, Caching.
Tree	Hierarchical structure composed of nodes (root, parent, child).	Traversal (In/Pre/Post-order), Search, Insert, Delete.	$O(\log n)$ (for Balanced Trees)	File systems, HTML/XML parsing, Expression parsing.
Binary Search Tree (BST)	A binary tree where the left child key is always less than the parent, and the right child key is always greater.	Same as Tree.	$O(\log n)$ (Avg.) / $O(n)$ (Worst)	Sorted data storage, efficient searching.
Heap	A complete binary tree that satisfies the Heap Property (Min-Heap: Parent < Children; Max-Heap: Parent > Children).	Insert, Extract-Min/Max.	$O(\log n)$ (Insert/Extract)	Priority Queues, Heap Sort, finding Kth smallest element.

Structure	Description	Key Operations	Time Complexity (Avg.)	Use Cases
Graph	A set of Vertices (nodes) and Edges (connections).	Traversal (BFS/DFS), finding Shortest Path.	$O(V+E)$ for traversal	Social networks, Maps/GPS routing, Network topology.

Part 3: Core Algorithms

1. Searching Algorithms

- Linear Search:** Checks every element sequentially until a match is found.
 - Complexity:* $O(n)$.
 - Best for:* Unsorted data or small lists.
- Binary Search:** Efficiently searches a **sorted** array by repeatedly dividing the search interval in half.
 - Complexity:* $O(\log n)$.
 - Best for:* Large, sorted datasets.

2. Sorting Algorithms

Algorithm	Average Time Complexity	Worst-Case Time Complexity	Key Idea
Bubble Sort	$O(n^2)$	$O(n^2)$	Repeatedly swap adjacent elements that are out of order.
Insertion Sort	$O(n^2)$	$O(n^2)$	Build the final sorted list one item at a time.
Merge Sort	$O(n \log n)$	$O(n \log n)$	Divide & Conquer: Splits list in half, recursively sorts, then merges the sorted halves.
Quick Sort	$O(n \log n)$	$O(n^2)$	Divide & Conquer: Picks a pivot and partitions the array into two sub-arrays around the pivot.

Algorithm	Average Time Complexity	Worst-Case Time Complexity	Key Idea
Heap Sort	$O(n \log n)$	$O(n \log n)$	Uses a Max-Heap to repeatedly extract the maximum element.

3. Graph Algorithms

- **Breadth-First Search (BFS):**
 - Traverses the graph **level by level** (explores all neighbors before moving to the next level).
 - Uses a **Queue** data structure.
 - *Use Case:* Finding the shortest path in an **unweighted** graph.
- **Depth-First Search (DFS):**
 - Traverses the graph as far as possible along one branch before **backtracking**.
 - Uses a **Stack** (or recursion).
 - *Use Case:* Detecting cycles, finding connected components, topological sorting.
- **Dijkstra's Algorithm:** Finds the **shortest path** from a starting vertex to all other vertices in a **weighted** graph with **non-negative** edge weights.
- **Kruskal's / Prim's Algorithms:** Used to find the **Minimum Spanning Tree (MST)**—a subset of the edges that connects all vertices together, without any cycles and with the minimum total edge weight.

4. Algorithmic Techniques

- **Recursion:** A function that calls itself to solve a problem by breaking it down into smaller, identical subproblems until a **base case** is reached.
- **Divide and Conquer:** Breaking a problem into non-overlapping subproblems, solving them independently, and combining the results (e.g., Merge Sort, Binary Search).
- **Greedy Approach:** Making the locally optimal choice at each step with the hope of finding a global optimum (e.g., Kruskal's/Prim's algorithms, Activity Selection Problem).
- **Dynamic Programming (DP):**
 - Used to solve complex problems by breaking them into simpler subproblems.

- Stores the results of subproblems to avoid recomputing them (**Memoization** or **Tabulation**).
- *Key properties:* **Optimal Substructure** (optimal solution contains optimal solutions to subproblems) and **Overlapping Subproblems**.
- *Examples:* Fibonacci Sequence, Knapsack Problem, Longest Common Subsequence.
- **Backtracking:** A variation of DFS used to find all or some solutions to computational problems, exploring partial solutions and abandoning a path as soon as it determines the path cannot lead to a valid full solution (e.g., N-Queens, Sudoku Solver).